🧠 DSA Problem Solving Framework (Ordered Thinking)

🔹 STEP 0: Problem Classification (MOST IMPORTANT)

Before touching code, ask:

- Is it about **array/string**?
- **Subarray / substring / contiguous?**
- **Sorting / ordering matters?**
- **Search / optimization?**
- **Graph / tree / recursion?**
- **Constraints ($n \le 10^5$ or $n \le 10^9$?)**

👉 This step decides EVERYTHING.

🔹 STEP 1: Brute Force Thinking (Always start here)

- Try all possibilities
- Nested loops / recursion

⏱️ Time: $O(n^2)$, $O(n^3)$

👉 WHY?

- Helps you understand constraints
- Often optimized later into sliding window / hashing

💡 Memory rule:

"If you don't know brute force, you don't know optimization."

🔹 STEP 2: Reduce Time using Hashing

Use:

- `unordered_map`
- `set`

When:

- Need **fast lookup**
- Frequency count
- Duplicate detection

👉 Examples:

- Two Sum
- Longest subarray with sum K

⏱️ Time: O(n)

💡 Memory rule:

"Whenever you see lookup → think hashing."

🔹 STEP 3: Two Pointer Technique

Use when:

- Array is **sorted** OR
- Need pair/triplet
- Opposite ends processing

👉 Examples:

- Pair sum in sorted array
- Container with most water

⏱️ Time: O(n)

💡 Memory rule:

"Sorted + pair problem = two pointers."

🔹 STEP 4: Sliding Window

Use when:

- **Subarray / substring**
- **Contiguous**
- Need max/min/length

👉 Types:

- Fixed window (size k)
- Variable window

👉 Examples:

- Longest substring without repeating characters
- Max sum subarray of size k

⏱️ Time: O(n)

💡 Memory rule:

"Contiguous + optimization = sliding window."

🔹 STEP 5: Prefix Sum / Difference Array

Use when:

- Range queries
- Subarray sum

👉 Examples:

- Subarray sum equals K
- Range sum queries

⏱️ Time: O(n)

💡 Memory rule:

"Repeated sum queries → prefix sum."

🔹 STEP 6: Binary Search (VERY POWERFUL)

Use when:

- Sorted data OR
- **Answer is monotonic**

👉 Trick:

Even if array not sorted → search on answer

👉 Examples:

- Allocate books
- Minimum capacity

⏱️ Time: O(log n)

💡 Memory rule:

"Monotonic answer → binary search."

🔹 STEP 7: Sorting + Greedy

Use when:

- Need optimal choice step-by-step

👉 Examples:

- Activity selection
- Merge intervals

⏱️ Time: O(n log n)

💡 Memory rule:

"Sort + local best choice = greedy."

🔹 STEP 8: Heap / Priority Queue

Use when:

- Need **top K**
- Min/max repeatedly

👉 Examples:

- K largest elements
- Merge K sorted arrays

⏱️ Time: O(n log k)

💡 Memory rule:

"Repeated min/max → heap."

🔹 STEP 9: Recursion → Backtracking

Use when:

- All combinations / subsets

👉 Examples:

- Permutations
- N-Queens

⏱️ Time: Exponential

💡 Memory rule:

"Try all possibilities → backtracking."

🔹 STEP 10: Dynamic Programming

Use when:

- Overlapping subproblems
- Optimal substructure

👉 Examples:

- Knapsack
- Longest common subsequence

💡 Memory rule:

"Repeat work + optimization → DP."

🔹 STEP 11: Graph Algorithms

Use when:

- Nodes + connections

👉 Tools:

- BFS / DFS
- Dijkstra
- Union-Find

💡 Memory rule:

"Relationship problems → graph."

🔹 STEP 12: Advanced Tools

Use when needed:

- **Trie** → strings/prefix
- **Segment Tree** → range queries + updates
- **Fenwick Tree** → prefix queries
- **Quick Select** → kth smallest/largest
- **Bit Manipulation** → XOR tricks

🔥 FINAL INTERVIEW FLOW (Golden Strategy)

When question comes:

1. **Understand constraints**
2. **Try brute force**
3. Ask:
    - Can I use hashing?
    - Is it sorted → two pointer?
    - Is it contiguous → sliding window?
    - Is answer monotonic → binary search?
4. Optimize step-by-step

⚠️ Common Mistake (YOU mentioned this)

> "random thinking"

Fix:
👉 Always follow this order:

```
Brute → Pattern identify → Optimize → Code
```

🚀 Bonus: Built-in Functions You Should Know

- `sort()` → O(n log n)
- `lower_bound()` / `upper_bound()`
- `next_permutation()`
- `accumulate()`
- `binary_search()`

👉 These save time in interviews.

🧩 Final One-Line Master Rule

"Identify pattern first, then pick tool — never pick tool randomly."

↓